PYTHON FUNDAMENTALS

Variables, types, conditionals, and expressions

ale66

INTRODUCTION

PROGRAMMING WITH PYTHON

- simple & *chic* programming language; name comes from *Monty Python*
- easy: reading code is almost like reading English
- simplicity as a result of design:
  - focus on the problem, not on the language or on how data sits in memory
  - a way to represent an easy-to-read program
- free, open source, well-supported and maintained: VS Code, Jupyter, PyPI, Anaconda, Docker…

PROGRAMMING, ONE INSTRUCTION AT A TIME

- a *high-level* language: a fixed alphabet and grammar to describe instructions
- a program is a sequence of instructions
- an interpreter is software that reads progrms and maps each instruction into low-level commands
- the OS system maps low-level into executable commands for the specific hardware underneath

even adding two numbers requires visibility of RAM and the CPU:

```
1  total_price =  price + vat
```

```
1  load the number from memory location 2001 into the CPU
2  load the number from memory location 2003 into the CPU
3  add the two numbers in the CPU
4  store the result into location 2005
```

COMPILER OR INTERPRETER?

Compiler: creates a new executable file, e.g., *my_app.exe*

- fast, efficient execution
- needs an ad hoc user interface
- specific to the hardware platform

Interpreter

- Python default mode
- executes the program inside its own memory
- easier to set up and control (chat with Python)

SIMPLE OUTPUT

SIMPLE PYTHON COMMANDS

```
1  # Command to print number 10!
2  print(10)
```

- comments start with #
- print writes to the screen

```
1  # find the result then print it
2  print(10 + 20)
3  print(10 + 2 * 20)
4  print((2 * 4) + 6)
5  print(2 ** 2)
```

PRINTING TEXT

```
1 # prints on the screen:
2 print('Have a nice Autumn term!')
3 print()
```

text is delimited by single (`'...'`) or double (`"..."`) quotes.

Double quotes are needed when `'` is used inside the text, called *string*

each `print` corresponds to a new line of the output interface (the terminal)

```python
# Let's print two names
print('Ale')
print('Pro')
print()

# Or
print("Ale", "Pro")
# Or
print('Ale', 'Pro')
# Or
print('Ale ' + 'Pro')
```

B: VARIABLES

PYTHON VARIABLES

programs manipulate data that sits in memory (RAM)

variable: a string name for a memory area (think Excel cells)

our variables are akin to both parameters and variables of mathematics

a funny = symbol assigns a value to the variable *on its left*

```
1  name = 'Nik'
2  age = 24
```

MORE EXAMPLES

```
# extract the value then print it
print(name)
print(name, age)
print(name, ', ', age)
```

```
x = age + 10

print(x)
print(10 / x)        # beware of division by 0!
print(2 * x + y)     # what does it print?
```

VARIABLES CAN CHANGE THEIR CONTENT

an **assignment statement**, not a mathematical equation:

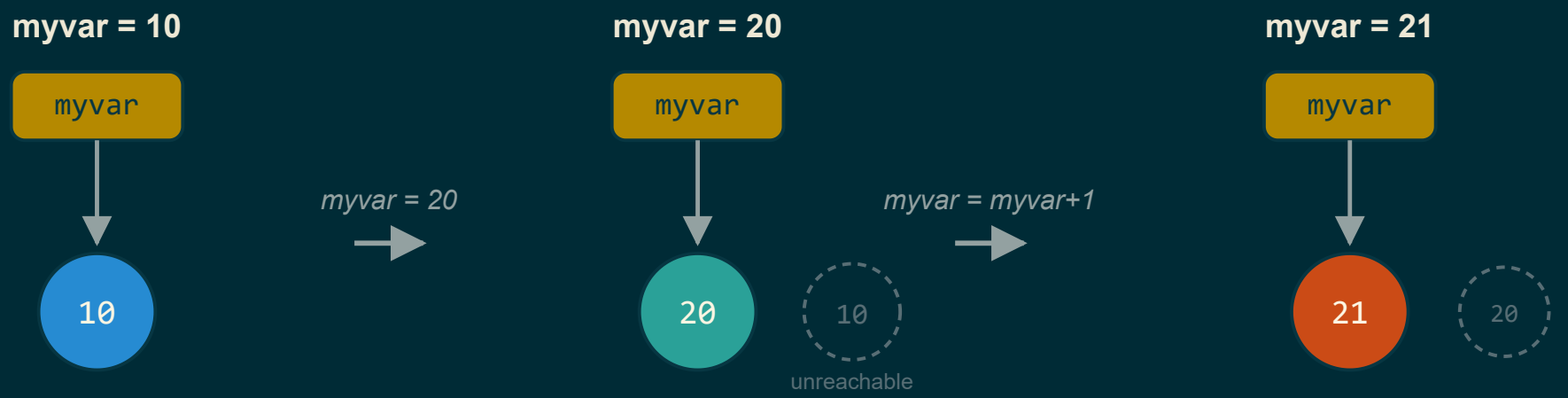
```
fahrenheit = 9 / 5 * celsius + 32
```

```
# This is an assignment expression!
myvar = 10
print(myvar)

myvar = 20              # assign a new value
print(myvar)

myvar = myvar + 1
print(myvar)            # prints 21
```

HOW MEMORY LOOKS LIKE



The name moves to a new value; it does not change the old one

- assignment rebinds the **name** to a new value; it does not modify the old value in place

INPUT

reading from the keyboard: `enter`/`return` ends the input

```
1 input()
```

assign the input to a variable: `<variable> = input(<prompt>)`

```
1 # This is an input example
2 name = input("Please enter a name: ")
3 print(name)
4
5 # This is another input example
6 my_fav_num = input("What is your favorite number? ")
7 print("Your favorite number is ", my_fav_num)
```

MORE THAN ONE INPUT

```
1  # This is another input example
2  fName = input("Give first name: ")
3  lName = input("Give surname: ")
4  print(fName, lName)
```

every call to `input()` reads one line and always returns **text**

keep that in mind — it is the subject of the next section

QUIZ 2: FILL THE GAPS

a. `_____` `# print your name`

b. `_____` `# print number 10`

c. `_____` `# print the sum of 1 and 2`

Solutions

```
1  a) print("Ale")   # print your name
2  b) print(10)           # print number 10
3  c) print(1+2)           # print the sum of 1 and 2
```

QUIZ 3: FILL THE GAPS

```
1  # Provide the command to prompt the user to enter a name
2  my_input = d) _________________("Enter your name: ")
3
4  # Provide the command to print the variable my_input
5  e) _________________(my_input)
```

Solution

```
1  my_input = d) __input__("Enter your name: ")
2  e) __print__(my_input)
```

C: TYPES

DATA TYPES

- **Integers**: `x=10, y=100 ...`
- **Floats**: `pi=3.14, a=10.10, rate=0.0992929`
- **Strings** (text): `name='Ale', age='32'`
- **Boolean** (`True` or `False`, `1` or `0`): `headphones=True`

WORKING WITH NUMBERS

Python sees all keyboard input as **text**

to do arithmetic, convert text into a numerical representation: int(eger) or float or complex

```
 1  # The input is 10
 2  x = input("Give a number")
 3  print(x)
 4  # x is text, even though it looks like a number
 5  # This will print 10!
 6
 7  # This is another input
 8  x = input("Give a number")
 9  print(x + 20)
10  # Error! Python does not know that x is a number...
```

WHAT IS THE OUTPUT?

```
1  # This is an input
2  x = input("Give a number")
3  print(x * 2)
```

What does it print if the user enters `10`?

- `20`
- an error...
- `1010` ← **correct**: `x` is text, so `*2` repeats it

CONVERTING: STRING TO INTEGER

use `int(<value>)`

```
1  # This is another input example
2  x = input("Give a number")
3  print(int(x) * 2)
```

What does it print if the user enters `10`? **`20`**

```
1  # Hint: if you expect an integer, convert the input directly
2  x = int(input("Give an integer number"))
3  print(x * 2)    # Will print 20!
```

CONVERTING: STRING TO FLOAT

use `float(<value>)`

```
1  # This is another input example
2  x = input("Give a number")
3  print(float(x) + 2)
```

What does it print if the user enters `10.5`? **`12.5`**

CONVERTING: INT TO STRING

use `str(<value>)`

```
1  x = 10
2  print("ID" + x)          # error: can't add str and int
3  print("a" + str(x))      # correct: "a10"
```

RECAP

- basic data types: `Integer`, `Float`, `Complex`, `String` (text), `Boolean`
- converting types: `int(<value>)`, `float(<value>)`, `str(<value>)`

QUIZ 4

```
1  a = input("give a number")
2  # The user enters 10
3  b = input("give a second number")
4  # The user enters 20
5  print(a + b)
```

What is the output?

Solution

If the input is `10` then `20`, this prints `1020` — both are still text.

```
1  a = int(input("give a number"))     # 10
2  b = int(input("give a second number"))  # 20
3  print(a + b)   # This will print 30!
```

D: CONDITIONAL EXECUTION

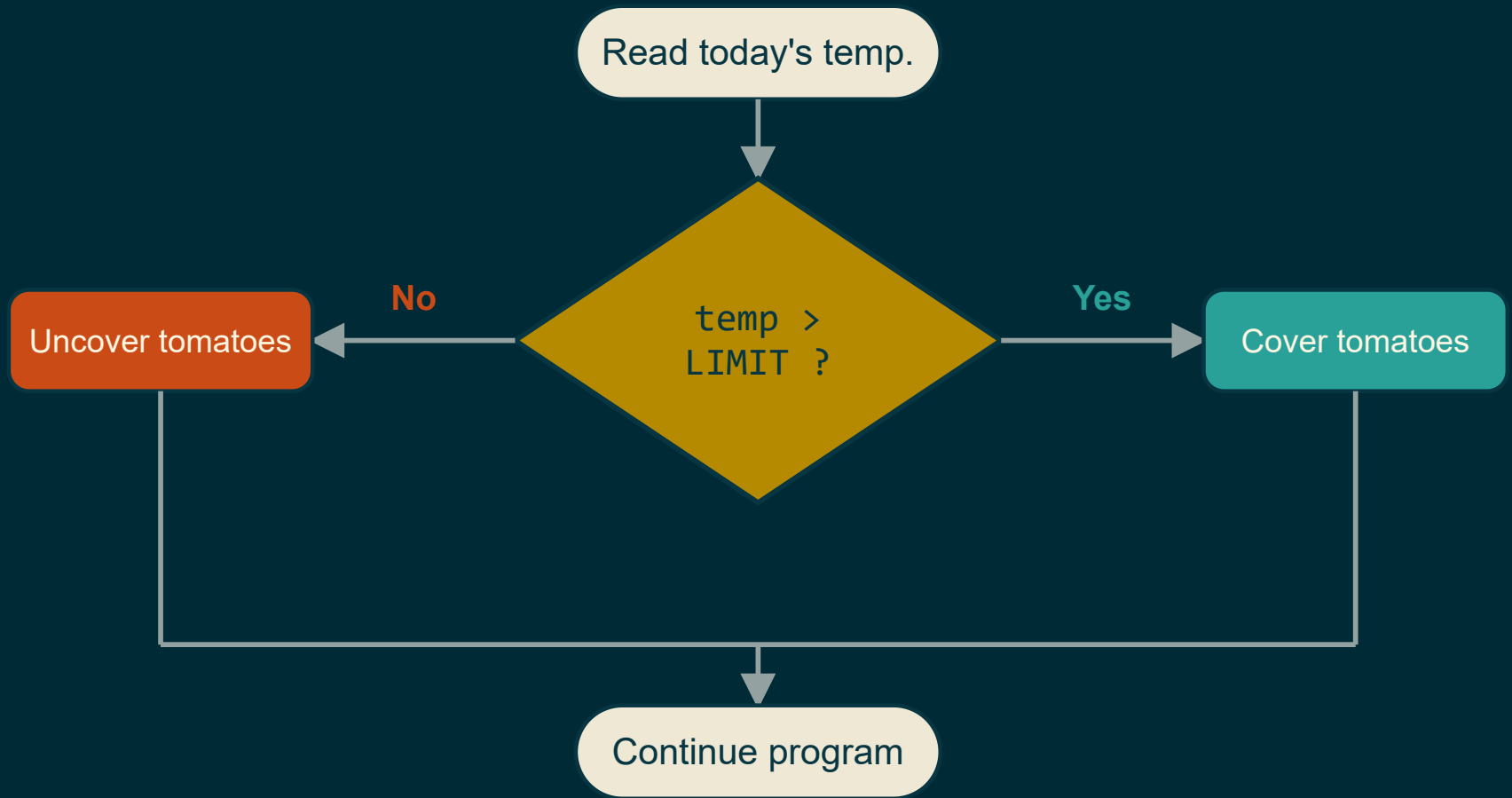
CONDITIONAL EXECUTION

So far, every instruction in our program (a `.py` file) has run from top to bottom

Next, execution will be driven by the actual input data — a **decision diagram**

here decision means evaluating a `True`/`False` question; some parts of the code might never run

```python
LIMIT = 32
temp = int(input("Temp. Today?"))
if temp > LIMIT:
    # Notice the indentation
    print("Go cover the tomatoes!")
```



PYTHON CONDITIONALS

operator	meaning
`a == b`	equals
`a != b`	not equals
`a < b`	less than
`a <= b`	less than or equal to
`a > b`	greater than
`a >= b`	greater than or equal to

```
1  a = 10
2  b = 9
3  if b < a:
4      # Notice the indentation
5      print("b is less than a")
```

github.com/ale66/learn-web

CONDITIONALS: IF/ELSE

Syntax:

```
if <condition>:
    <statement>
else:
    <statement>
```

```
a = 10
b = 9
if b < a:
    print("b is less than a")
else:
    print("a is greater than b or equal")
```

3-WAY CONDITIONALS

Syntax:

```
if <condition>:
    <statement>
elif <condition>:
    <statement>
else:
    <statement>
```

```
a = 10
b = 9
if b < a:
    print("b is less than a")
elif b > a:
    print("b is greater than a")
else:
    print("a and b are equal")
```

QUIZ 6: FILL THE GAPS

Print `"Hi"` if `a` is greater than or equal to `b`.

```
1  a = 22
2  b = 82
3  _________ a _________ b _________
4      print("Hi")
```

Solution

```
1  a = 22
2  b = 82
3  if a >= b:
4      print("Hi")
```

QUIZ 7: LAUNCH A VS CODE

prompt the user for a number then print:

- `'Positive'` if the number is greater than 0
- `'Negative'` if the number is less than 0
- `'Zero'` otherwise

Solution

```python
user_input = int(input('Enter a number: '))

if user_input > 0:
    print('Positive')
elif user_input < 0:
    print('Negative')
else:
    print('Zero')
```

GENERAL SUMMARY

- compiler vs. interpreter: Python is a line-by-line interpreter
- variables store data: name / type / value
- use `input()` to get data from the user, `print()` to output it
- data types: Integers, Floats, Strings (Text), Boolean
- `int()` converts text into numerical data; `float()` and `char()` work similarly
- `if` / `elif` / `else` create conditional, data-driven execution

E: EXPRESSIONS

RECAP QUIZ

```python
# 1
a = input("give a number")   # user inputs 10
b = input("give a number")   # user inputs 10
print(a + b)
# What does it print?
```

```python
# 2
a = input("give a number")   # user inputs 10
b = input("give a number")   # user inputs 10
print(int(a) + b)
# What does it print?
```

SOLUTIONS

```
# 1
print(a + b)
# What does it print?
1010
```

```
# 2
print(int(a) + b)
# What does it print?
ERROR!
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

LOGICAL OPERATORS

- **equal**: `10 == 10` → `True`; `if (a == 10): ...` tests a condition; `a = 10` *assigns* a value, it is not a condition
- **not equal**: `5 != 5` → `False`; `if (a != 0): print(10/a)`

```
1  if (temp > 30) and (distance_to_beach < 50):
2      print("Let's go to the beach!")
3
4  if (raining == True) or (day == "Sunday"):
5      tea == True
```

BOOLEANS

A Boolean (or logical) expression evaluates to one of two states, `True` or `False`. Python's `bool` type holds one of them.

```
1 x = int(input("Give a num: "))
2 enterTheIf = False
3 if x == 10:
4     print("This is true!")
5     enterTheIf = True
6 print(enterTheIf)
```

```
1 # Another example!
2 a = int(input("Give a num: "))
3 isZero = False
4 if a == 0:
5     isZero = True
6 if isZero == False:
7     print(100 / a)
```

AWKWARD RUN 1 — WHAT'S WRONG?

```python
age = int(input("Enter your age "))

if (age < 10):
    print("You pay 5")
elif (age > 10 or age < 60):
    print("You pay 10")
elif (age > 60):
    print("You pay 4")
```

```
Enter your age 70
You pay 10
```

please wait for all to finish

AWKWARD RUN 2 — WHAT'S WRONG?

Same code, different input:

```
1  Enter your age 10
2  You pay 10
```

please wait for all to finish

SOLUTION

The condition should be `and`, not `or`:

```
1  age = int(input("Enter your age "))
2
3  if (age <= 10):
4      print("You pay 5")
5  elif (age > 10 and age <= 60):
6      print("You pay 10")
7  elif (age > 60):
8      print("You pay 4")
```